

NHL Shots Model

The National Hockey League represents the highest level of competition in ice hockey. Every player on Team USA, the gold medalists of the 2026 Olympics, was playing in the NHL when the tournament took place. When compared to other major professional sports, the NHL lacks much of the high level analytics that are currently reshaping how games like basketball, football, and baseball are played. In this project, I will build a model that starts to lay some of the groundwork for understanding what factors play into the success of an NHL team, starting with shots. This project focuses on modeling goal outcomes in NHL shot data, with particular emphasis on handling class imbalance, evaluating model performance, and comparing different machine learning approaches. While the primary objective was technical, if small adjustments are made to the model, it could have practical applications such as aiding coaching staff in setting up plays, using aggregated goal probabilities to generate an expected goals statistic, and provide insight into the individual skills of players/goalies relative to the expected outcomes provided by the model.

Data

This project used over 1 million shots taken in the regular season between 2015 and 2024 with over 100 engineered features to capture spatial, temporal, and contextual data. This data was pulled from [moneypuck.com](https://www.money puck.com).

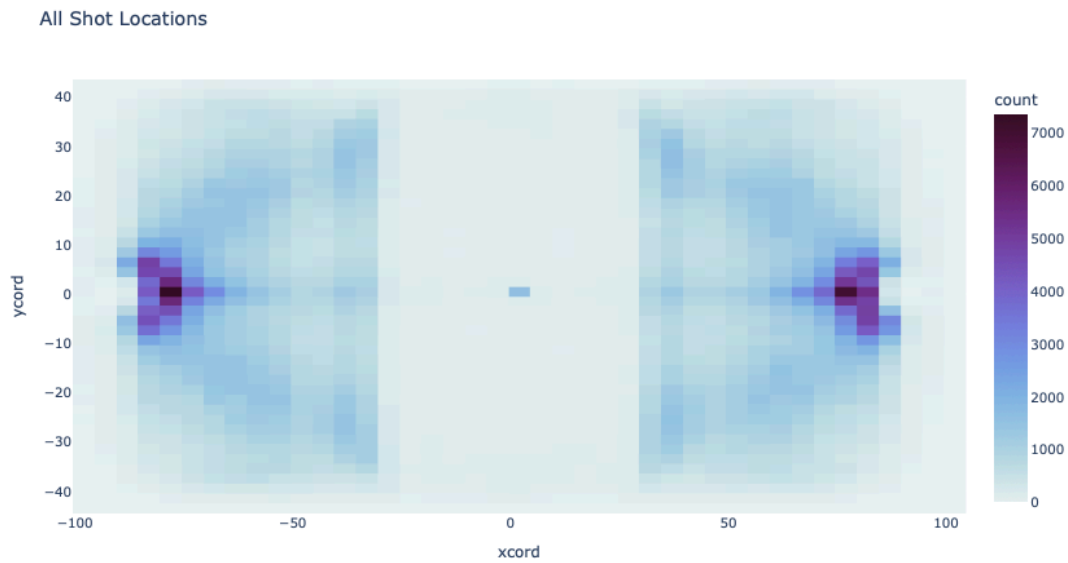
Cleaning

Because the aim of this project is to predict the outcome of a shot, substantial preprocessing was necessary to ensure consistency and accuracy. Outside of normalizing naming conventions and adjusting the data types of variables, there were a few more notable challenges that I faced while cleaning the data. This dataset included a lot of contextual information for every shot. This meant that there were a lot of columns that needed to be dropped to prevent data leakage, including some less obvious features that only became apparent after early modeling. Rare categories that required grouping prior to encoding to prevent too many columns. Some features (notably those that include “timeonice”) had already had missing values filled, although the filled values presented significant outliers. After identifying these features I was able to adjust the values to be less disruptive.

EDA

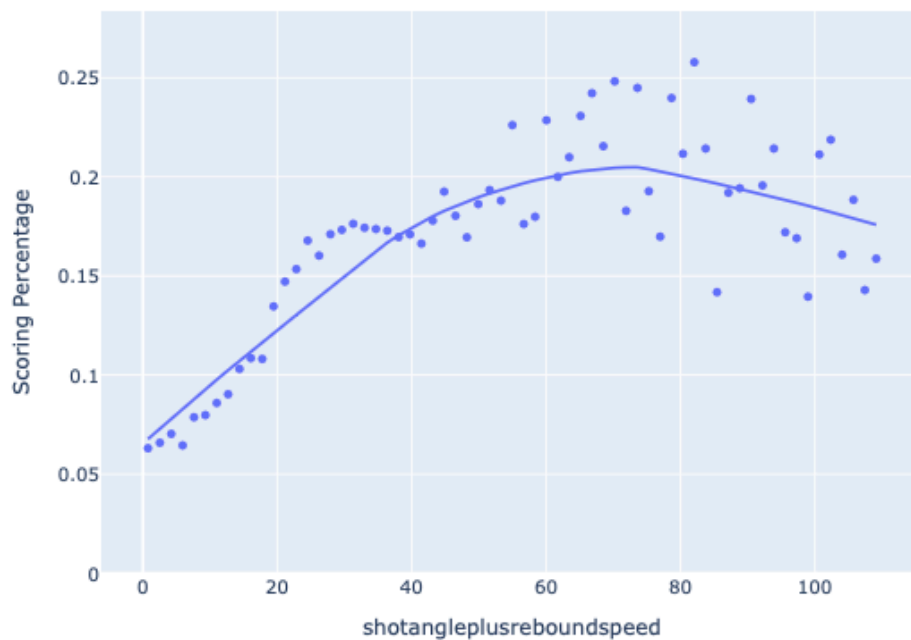
Through EDA I found spatial, temporal, and contextual data were all significant when determining the likelihood of a shot resulting in a goal.

Spatial - X/Y coordinates of shot



Temporal - Time between events

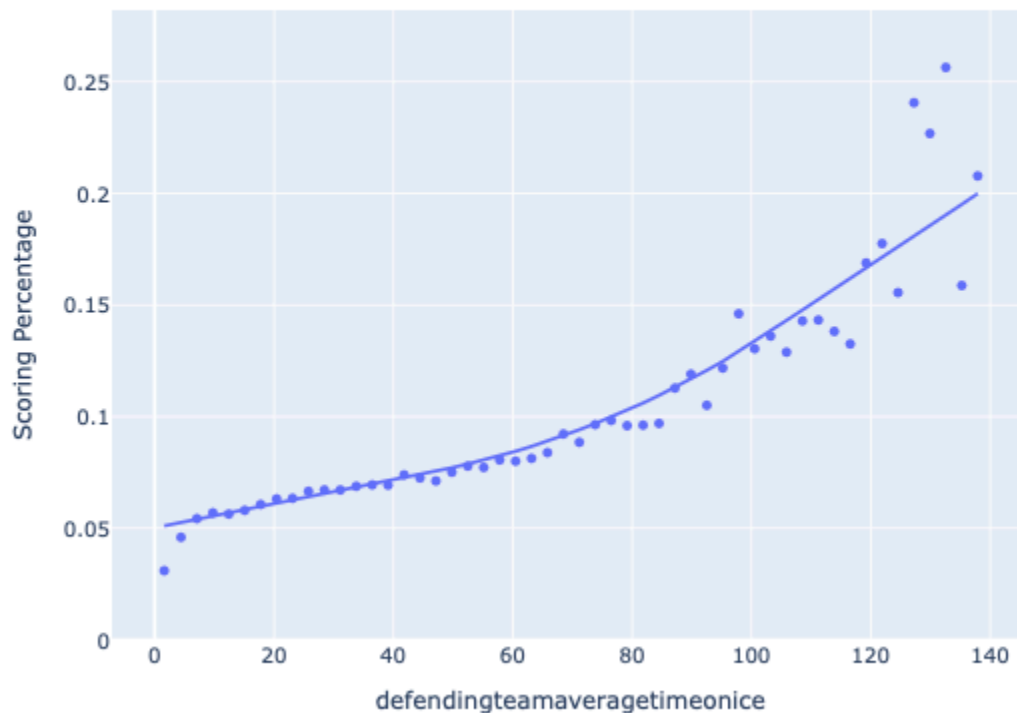
Scoring percentage given shotangleplusreboundspeed



Shot angle plus rebound speed is an engineered feature that combines the change in angle from one shot to another, and the time between the two shots.

Contextual - Shot type, players on ice, player time on ice, etc.

Scoring percentage given defendingteamaveragetimeonice

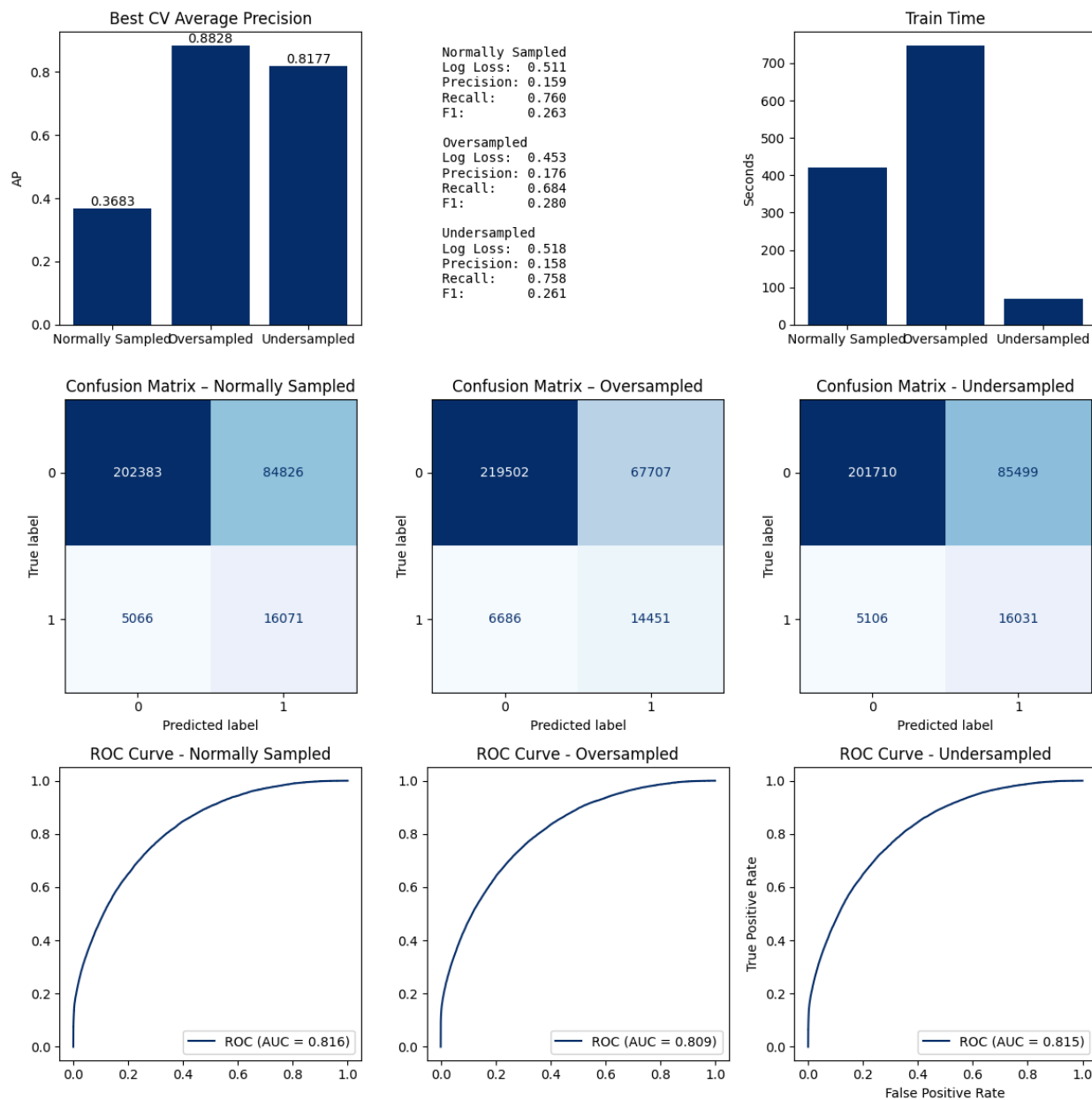


Contextual data made up a majority of the features used in this model

Modeling

Initial model selection focused on comparing fundamentally different model classes: linear models (logistic regression), bagging models (random forest), and boosting models (XGBoost). At this point, given the strong class imbalance, I used PR AUC as my primary metric. Through testing, I found that XGBoost significantly outperformed the other two models, which led me to move forward with other boosting models.

It was at this point that I tested oversampling and undersampling the data, as the dataset I started with was heavily imbalanced (~7% of shots were goals).

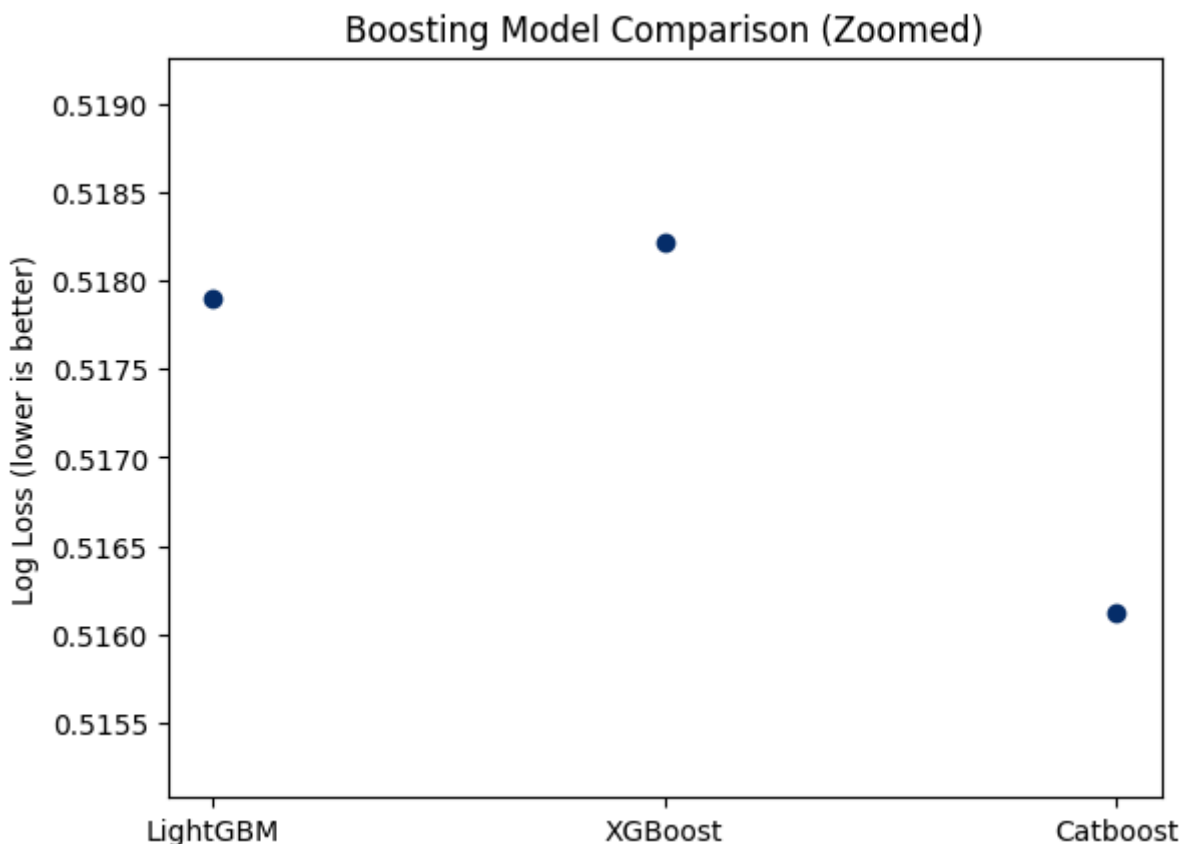


Oversampling performed better than undersampling when looking at CV Average Precision (normal skewed by imbalanced dataset), and outperformed both regular sampling and undersampling in log loss and F1 scoring. Undersampling decreased the AUC slightly, but to a lesser degree. Given the significantly decreased train time when undersampling, I used undersampling for the remaining model selection and hyperparameters tuning, with the intention of using oversampling for final model selection.

In my second round of modeling, I used three different boosting models:

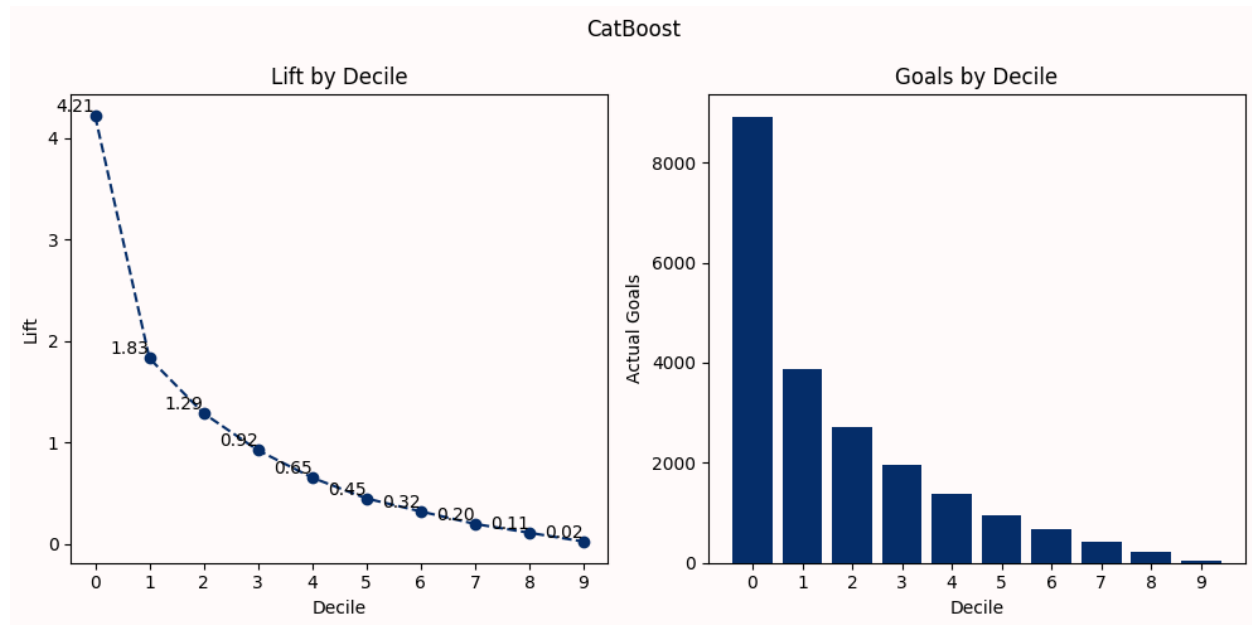
- XGBoost - Strong baseline that is well documented
- LightGBM - Computational efficiency and ability to scale to large datasets
- CatBoost - Handling of categorical variables

Now working with a balanced dataset, I switched my primary scoring parameter to log loss, as it served as a more standard and consistent scoring method for the models I was using (CatBoost does not natively support average precision) Between XGBoost, CatBoost, and LightGBM, CatBoost slightly outperformed XGBoost and LightGBM, with the best iteration having a test log loss mean of 0.516124.



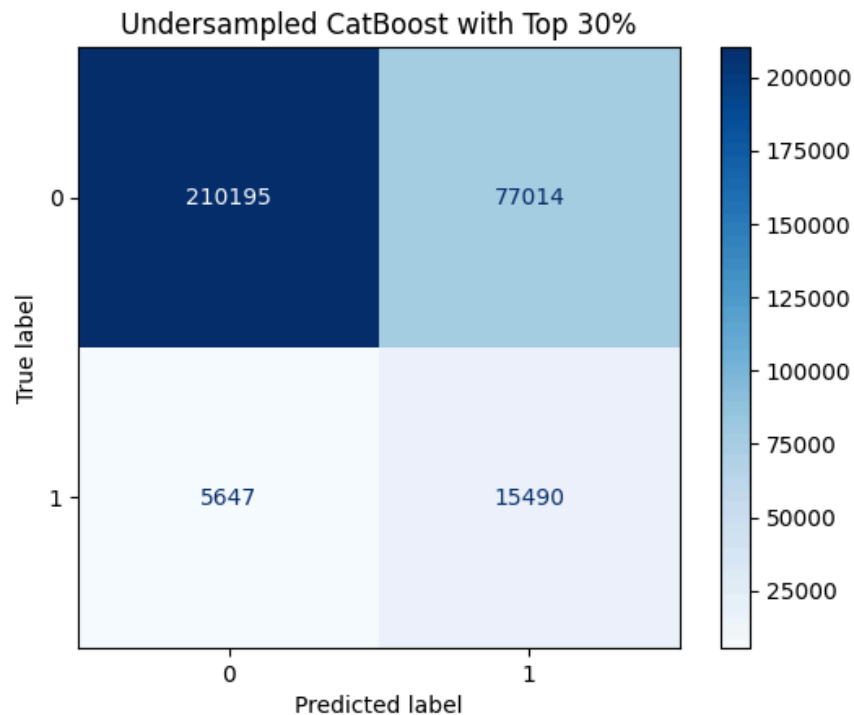
Decile Evaluation

At this point, with the default threshold set to 0.5, the model was significantly over-predicting the number of goals, resulting in far too many false positives. Because I have no reason to strongly prefer accuracy or precision, I ended up breaking the data into deciles, and from there using the lift score of each decile to determine the threshold.



Using the values displayed in the above figures, I set the threshold to 0.7, predicting the shots in the top deciles (0-2) to be goals. This step also provided valuable insight into the performance of the model, with the top decile having a lift score of 4.21.

After running the oversampled catboost search and finding that the oversampled data actually worsened overall model performance, the undersampled CatBoost with a 30% Threshold was selected for the final model, with the confusion matrix for that model seen below:



Future Work

Given more time, I would like to expand this to better understand how luck and momentum play a role in NHL games, including player/team performance in the predictions. Doing a time-series analysis looking at games and trends over the course of a season would allow me to track team and player performance at a more aggregate level. On the other hand, I could also attempt to push this model to function more as a real time prediction system. On a more practical note, the hyperparameter tuning ate up significant portions of time, with my single oversampled CatBoost grid search taking over 8 hours to complete. With more resources available, I could potentially expand my search and increase the number of iterations to improve model performance.

Credit

All data used in this project was pulled from moneypuck.com.